Call Log Management Using Doubly Linked List

Problem Statement and Technical Details

1. Problem Statement

A telephone system keeps a log of calls, where each entry records the caller's name and the call duration in minutes. The task is to design and implement a program that stores two separate call logs using a **doubly linked list**, merges them into a single log, and then supports displaying the log in both directions, searching for a caller, deleting a caller's entry, and reporting call statistics. All memory used by the lists must be released before the program ends.

2. Objectives

- Model each call record as a node with a caller name, a duration, and two pointers (next and prev).
- Build two call logs from user input by appending records at the end.
- Merge the second log onto the end of the first, keeping the links correct in both directions.
- Traverse the merged log forward and backward.
- Search for a caller and report the matching record.
- Delete a caller's record and repair the links of its neighbours, including the head and tail cases.
- Compute the total number of calls and the total call duration.
- Free all dynamically allocated nodes.

3. Data Structure

Each node of the list is defined as follows:

```
struct Node
{
    string name;     // caller name
    int duration;    // call duration in minutes
    Node* next;      // pointer to the next call
    Node* prev;      // pointer to the previous call
};
```

The list is accessed through a head pointer. The head's *prev* pointer and the tail's *next* pointer are NULL, so the list has no circular links.

4. Functions

Function	Purpose
createNode(name, duration)	Allocates a new node, sets its fields, and sets both links to NULL.
insertEnd(head, name, duration)	Appends a new call at the end of the list. Updates head if the list is empty.
displayForward(head)	Prints every call from the head to the tail. Prints a message if the list is empty.
displayBackward(head)	Walks to the tail, then prints calls from the tail back to the head using prev.
mergeLists(head1, head2)	Links the tail of list 1 to the head of list 2 and returns the combined head.
search(head, name)	Returns a pointer to the first node whose name matches, or NULL.
deleteNode(head, name)	Removes the first matching node, updating head or the neighbouring links.
totalDuration(head)	Returns the sum of all call durations.

Function	Purpose
countCalls(head)	Returns the number of nodes in the list.
deleteList(head)	Deletes every node and sets head to NULL.

5. Program Flow

- Read the number of calls in the first log, then read each caller name and duration and append it to list 1.
- Repeat the same process for the second log (list 2).
- Display list 1, list 2, and the merged list.
- Display the merged list forward and backward.
- Read a caller name, search for it, and display the result or a not-found message.
- Read a caller name, delete it, and display the updated log.
- Display the total number of calls and the total duration.
- Delete the whole list and exit.

6. Input and Output Format

Input: an integer n, followed by n pairs of (name, duration); then an integer m, followed by m pairs; then a name to search for; then a name to delete. Names and durations are read with *cin* >>, so each name must be a single word without spaces.

Output: the logs at each stage, the search result, the result of the deletion, and the statistics.

7. Worked Example

Input:

```
n = 2:  Alice 5,  Bob 3
m = 1:  Carol 7
Search: Bob
Delete: Bob
```

Expected results:

- Merged log: Alice - 5 min, Bob - 3 min, Carol - 7 min.
- Backward display: Carol - 7 min, Bob - 3 min, Alice - 5 min.
- Search for Bob: Caller Bob, Duration 3 minutes.
- After deleting Bob: Alice - 5 min, Carol - 7 min.
- Statistics after deletion: Total Calls 2, Total Call Duration 12 minutes.

8. Expected Complexity

- insertEnd, displayBackward, mergeLists, and search: O(n) time, because the list must be traversed. insertEnd and mergeLists can be made O(1) if a tail pointer is kept.
- deleteNode: O(n) because it searches for the node first; the unlinking itself is O(1).
- displayForward, totalDuration, countCalls, and deleteList: O(n) time, O(1) extra space.

9. Observations and Possible Improvements

- **Duplicate names:** search and deleteNode act only on the first match, so a second entry with the same name remains.
- **Tail pointer:** insertEnd and mergeLists walk the whole list every time. Keeping a tail pointer would make appending O(1).
- **Redundant display:** the merged log is printed twice in main (once as MERGED CALL LOG and again as FORWARD DISPLAY).
- **Input with spaces:** cin >> stops at whitespace, so names such as "Mary Ann" cannot be entered. getline could be used.
- **Input validation:** negative durations, negative counts, and non-numeric input are not checked.
- **Merge with an empty list:** the merge handles NULL inputs correctly; the first list's head is returned when the second list is empty.
- **Memory:** deleteList frees only list 1 after merging, since list 2's nodes are now part of list 1. This is correct and avoids a double free.

10. Conclusion

The program demonstrates the core operations of a doubly linked list: insertion at the tail, bidirectional traversal, merging two lists, searching, deletion with pointer repair, aggregate computation, and proper memory cleanup. It is a suitable exercise for understanding pointer manipulation and dynamic memory management in C++.